Closed-Loop Threat-Guided Auto-Fixing of Kubernetes Container Security Misconfigurations Supplemental Appendices

How to Read This Appendix (Plain English)

This appendix keeps the explanations simple and points to the exact files that back every claim. Tables sit directly under their section titles so you can skim them quickly.

A Grok/xAI Failure Analysis

The raw data for the Grok/xAI failure analysis can be found in `data/grok_failure_analysis.csv`. This file provides a comprehensive list of all failure causes and their corresponding counts, generated from the analysis of the 5,000-manifest Grok corpus.

B Risk Score Worked Example

The released telemetry enables reviewers to recompute risk units and $\Delta R/t$ for any queue item. As a concrete example we trace detection `001` from the Grok/xAI replay:

1. Look up the detection metadata in `data/batch_runs/detections_grok200.json` to confirm the violation is `latest-tag`.
2. Normalise the policy identifier and pull its risk weight and expected latency from `data/policy_metrics_grok200.json`. For `no_latest_tag` the risk is 50 units and the proposer+verifier expected time is 9.363 s (averaged from the recorded latencies).
3. Inspect the proposer/verifier records (`data/batch_runs/patches_grok200.json`; `data/batch_runs/verified_grok200.json`) to see that the patch was accepted with a measured end-to-end latency of 7.339 s and verifier latency of 0.332 s.

Because the patch succeeded, the pre-risk $R^{\text{pre}} = 50$ drops to $R^{\text{post}} = 0$, yielding $\Delta R = 50$ and $\Delta R/t = 50/9.363 = 5.34$ risk units per second. Summing the same quantities across the corpus reproduces the risk-calibration table in the manuscript, as computed by `scripts/risk_calibration.py`.
Plain version. We look up the finding, note its "risk points" and expected time to fix, check that the patch actually worked, and then see that the risk points went from 50 down to 0. That is the same math used for every item in the tables.

C Acronym Glossary

Acronym	Definition
CIS	Center for Internet Security
PSS	Pod Security Standards
CRD	Custom Resource Definition
RBAC	Role-Based Access Control
CTI	Cyber Threat Intelligence
KEV	CISA Known Exploited Vulnerabilities
EPSS	Exploit Prediction Scoring System
CVE/CVSS	Common Vulnerabilities and Exposures / Scoring System
RAG	Retrieval-Augmented Generation
MTTR	Mean Time To Remediate
CEL	Common Expression Language (Kubernetes)
SAST	Static Application Security Testing
DAST	Dynamic Application Security Testing

D Artifact Index

Table 1: Primary artifacts bundled with the paper.

Artifact Path	Description
`data/live_cluster/results_1k.json`	Live-cluster replay outcomes (1,000 manifests, dry-run/live apply parity).
`data/batch_runs/grok_5k/metrics_grok5k.json`	Grok/xAI telemetry (acceptance, latency, token counts) for the 5k sweep.
`data/risk/risk_calibration.csv`	Risk accounting summary (ΔR, residual risk, $\Delta R/t$) for supported and 5k corpora.
`data/metrics_schedule_compare.json`	Queue replay statistics for FIFO vs. risk-aware schedulers (rank, wait, $\Delta R/t$).
`data/grok_failure_analysis.csv`	Grok failure taxonomy (dry-run retrievals, StatefulSet validation, etc.).

E Evaluation Artifact Manifest

Table 2 expands the artifact index with record counts and purposes for full reproducibility.

Table 2: Key evaluation artifacts with record counts and purposes for full reproducibility.

Artifact Path	Purpose	Count
data/live_cluster/results_1k.json	Live-cluster replay outcomes (dry-run + apply)	1,000
data/live_cluster/summary_1k.csv	Live-cluster aggregate statistics	1
data/batch_runs/grok_5k/metrics_grok5k.json	Grok-5k acceptance & token telemetry	5,000
data/batch_runs/grok_full/metrics_grok_full.json	Manifest slice (1,313) acceptance	1,313
data/batch_runs/grok200_latency_summary.csv	Proposer latency summary (Grok-200)	280
data/batch_runs/verified_grok200_latency_summary.csv	Verifier latency summary (Grok-200)	140
data/eval/significance_tests.json	Statistical significance tests (z-test; Mann–Whitney U)	12
data/eval/table4_counts.csv	Table 4 manifest counts per corpus	4
data/eval/table4_with_ci.csv	Wilson 95% confidence intervals	4
data/scheduler/fairness_metrics.json	Scheduler fairness (Gini, starvation)	830
data/scheduler/metrics_schedule_sweep.json	Scheduler parameter sweep results	16
data/risk/risk_calibration.csv	Risk reduction (ΔR) per corpus	2

F Corpus Mining and Integrity

What this section is. How we gathered the manifests and proved the dataset is intact.

ArtifactHub mining pipeline. Running the data collection helper[1] renders Helm charts directly from ArtifactHub using `helm template`, normalizes resource filenames, and writes structured manifests under data/manifests/artifacthub/. The script records fetch failures and chart metadata so regenerated datasets can be diffed against the published summary.

Corpus hashes. After manifests are rendered, `python scripts/generate_corpus_appendix.py` emits docs/appendix_corpus.md, a SHA-256 inventory of every manifest (including the curated smoke tests in data/manifests/001.yaml and 002.yaml). This appendix enables reproducibility reviewers to verify corpus integrity and trace individual evaluation examples back to their Helm chart origins. In short: the hash list is a tamper check and a map back to each source.

[1]Command: `python scripts/collect_artifacthub.py --limit 5000`. Full source: scripts/collect_artifacthub.py.